

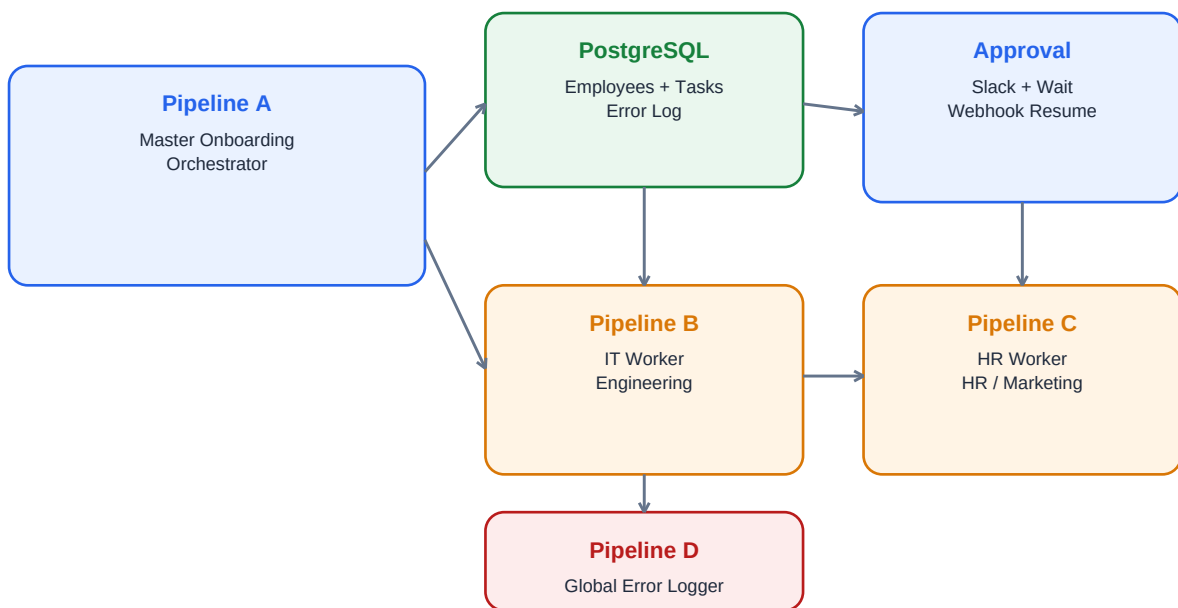
Production-Ready Employee Onboarding Automation

Documentation Package

Platform: n8n + PostgreSQL + Slack

Architecture: Master Orchestrator + Reusable Workers + Global Error Logger

Modular production onboarding architecture



Prepared for LMS submission

1. Executive Summary

This project implements a modular employee onboarding platform in n8n. A webhook receives onboarding requests, PostgreSQL validates duplicate email addresses and stores employee state, department routing invokes reusable provisioning workers, Slack requests manager approval, a Wait node resumes through a webhook, and a centralized error workflow records operational failures.

The supplied implementation contains Pipeline A as the master orchestrator, Pipeline B as the Engineering IT worker, Pipeline C as the HR/Marketing worker, Pipeline D as the global error logger, and a one-off PostgreSQL initialization workflow.

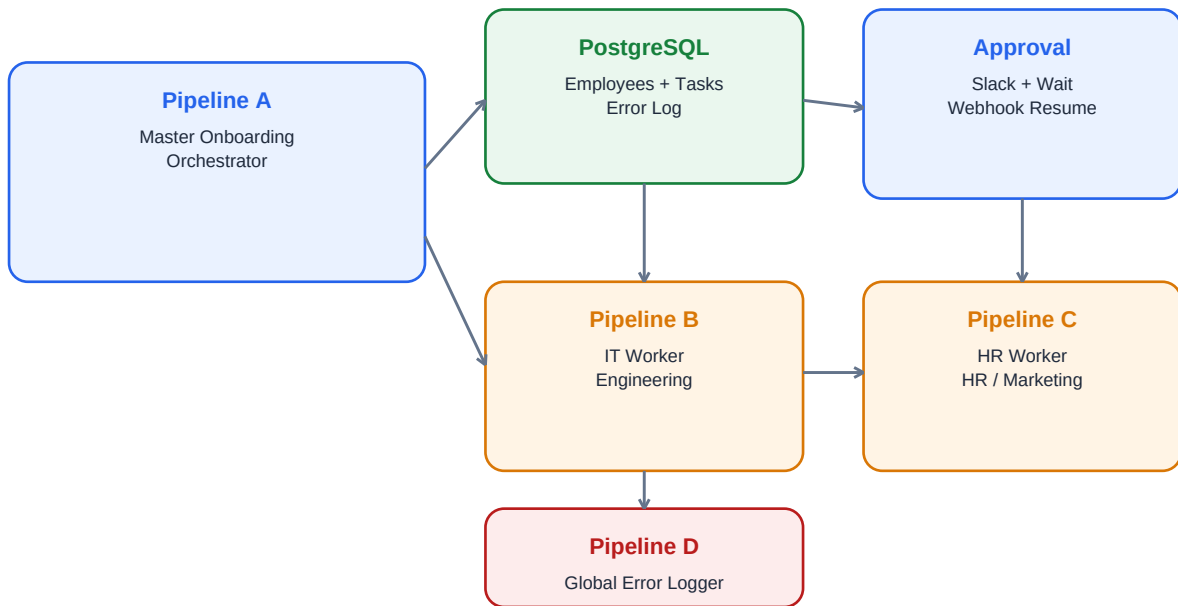
Assessment goals

- Prevent duplicate employee creation using email-based PostgreSQL validation.
- Create employee profile and onboarding checklist records.
- Route Engineering to IT provisioning and HR/Marketing to HR provisioning.
- Reuse child workflows through Execute Workflow inputs.
- Send a manager approval request through Slack and pause for a webhook response.
- Persist provisioning and approval state in PostgreSQL.
- Centralize workflow failures in a global error log.

Implementation evidence: Pipeline A contains the webhook, duplicate check, profile/checklist creation, department routing, worker invocation, Slack approval, Wait webhook resume, and approval update.

2. Workflow Architecture

Modular production onboarding architecture



The architecture separates orchestration from department-specific provisioning. Pipeline A owns onboarding state and human approval. Pipelines B and C are reusable workers. Pipeline D is configured as the centralized error workflow.

Pipeline relationships

Component	Responsibility	Interface
Pipeline A — Master	Ingress, validation, creation, routing, approval	POST webhook + Execute Workflow
Pipeline B — IT	Reusable Engineering provisioning	employee_id, department, role
Pipeline C — HR	Reusable HR/Marketing provisioning	employee_id, department, role
Pipeline D — Error	Centralized operational logging	Error Trigger → PostgreSQL

3. Pipeline A — Master Onboarding Orchestrator

Ingress

The workflow starts with a POST webhook at **/capstone-onboarding**. The incoming request supplies employee onboarding information such as `full_name`, `email`, `department`, and `role`.

Duplicate validation

`SQL_CheckDuplicates` queries `capstone_employees` by email. The `Is Duplicate?` decision sends an existing record to `Duplicate - Stop` and allows a new employee to continue.

Profile and checklist creation

- **SQL_CreateProfile:** inserts `full_name`, `email`, `department`, and `role` and returns `employee_id` and employee context.
- **SQL_CreateChecklist:** creates Account creation, Equipment allocation, Department provisioning, and Manager approval tasks.

Department routing

`Route_Department` sends Engineering to Pipeline B and HR/Marketing to Pipeline C. The worker inputs are mapped as `employee_id`, `department`, and `role`.

Slack approval and asynchronous wait

- Slack sends a manager approval request containing employee details.
- Wait For Approval resumes through POST webhook suffix **capstone-approval-response**.
- `SQL_UpdateApproval` writes the callback decision and `employee_id` to `capstone_employees`.

The supplied Pipeline A export confirms these stages and their connections.

4. Reusable Provisioning Workers

Pipeline B — IT Provision Worker

Pipeline B is a reusable child workflow invoked for Engineering employees. Its trigger accepts `employee_id`, `department`, and `role`. A simulation node represents GitHub/IT provisioning, followed by `SQL_UpdateITProvisioned`, which sets `it_provisioned = TRUE` for the employee.

Pipeline C — HR Provision Worker

Pipeline C is a reusable child workflow for HR and Marketing employees. It accepts `employee_id`, `department`, and `role`, simulates payroll and benefits setup, and then updates `hr_payroll_setup = TRUE` in `capstone_employees`.

Reusability benefits

- Provisioning logic is centralized instead of duplicated inside the master workflow.
- The master workflow passes a consistent employee context to child workflows.
- Additional departments can be added as new workers without rebuilding the entire orchestration flow.

Source implementation: Pipeline B and Pipeline C exports explicitly define their workflow inputs and PostgreSQL state updates.

5. PostgreSQL Database Design

The database initialization workflow creates three core tables: `capstone_employees`, `capstone_onboarding_tasks`, and `capstone_error_log`.

Table	Purpose	Important fields
<code>capstone_employees</code>	Employee master state	<code>employee_id</code> , <code>full_name</code> , <code>email</code> , <code>department</code> , <code>role</code> , <code>approval_status</code> , <code>it_provisioned</code> , <code>hr_payroll_setup</code>
<code>capstone_onboarding_tasks</code>	Onboarding checklist	<code>task_id</code> , <code>employee_id</code> , <code>task_name</code> , <code>status</code> , <code>created_at</code>
<code>capstone_error_log</code>	Operational error history	<code>workflow_name</code> , <code>workflow_id</code> , <code>execution_id</code> , <code>node_name</code> , <code>error_message</code> , <code>stack_trace</code>

Integrity controls

- `email` is UNIQUE and indexed for duplicate lookup.
- onboarding tasks reference `employee_id` through a foreign key.
- error records preserve workflow and execution context for troubleshooting.

6. Global Error Handling

Pipeline D — Global Error Logger is designed to be configured as the Error Workflow for Pipelines A, B, and C. It starts with an n8n Error Trigger and inserts execution context into `capstone_error_log`.

Captured diagnostics

- Workflow name and workflow ID
- Execution ID
- Last/failing node
- Error message
- Stack trace

The supplied Pipeline D export implements the Error Trigger → SQL_LogError connection and the INSERT into `capstone_error_log`.

Production troubleshooting practice

- Use n8n execution history together with the database error log.
- Keep credentials in n8n credential stores rather than hard-coding secrets.
- Apply appropriate database and execution-log retention policies.

7. Node Configuration Blueprint

Node / stage	Configuration / behavior
Webhook	POST /capstone-onboarding; receives employee payload.
SQL_CheckDuplicates	SELECT by email from capstone_employees.
Is Duplicate?	Existing email → Duplicate - Stop; new email → profile creation.
SQL_CreateProfile	Insert employee profile and return employee_id/context.
SQL_CreateChecklist	Create four onboarding tasks.
Route_Department	Engineering → IT; HR/Marketing → HR.
Invoke IT / HR Worker	Pass employee_id, department, role.
Slack Approval Request	Send manager approval request.
Wait For Approval	Webhook resume using capstone-approval-response; two-hour limit.
SQL_UpdateApproval	Update approval_status using employee_id.
Pipeline B SQL	Set it_provisioned = TRUE.
Pipeline C SQL	Set hr_payroll_setup = TRUE.
Pipeline D SQL	Insert error context into capstone_error_log.

8. Testing & Validation

The supplied screenshots show successful n8n executions for the master workflow and reusable worker workflows, including visible node input/output panels. The workflow exports provide direct configuration evidence for the core state transitions.

Test	Expected result	Evidence
New Engineering employee	Profile/checklist created; IT worker runs; it_provisioned becomes TRUE.	Pipeline A + B execution + DB row
New HR/Marketing employee	Profile/checklist created; HR worker runs; hr_payroll_setup becomes TRUE.	Pipeline A + C execution + DB row
Duplicate email	Duplicate - Stop path; no second employee profile.	Duplicate branch + DB verification
Approval callback	Wait resumes and approval_status updates.	Resume execution + DB row
Worker failure	Pipeline D records error context.	Error execution + error-log row

cURL validation template

```
curl -X POST "/webhook/capstone-onboarding" \ -H "Content-Type: application/json" \ -d '{"full_name":"Test Employee","email":"test@example.com","department":"Engineering","role":"Software Engineer"}'
```

Approval callback template

```
curl -X POST "" \ -H "Content-Type: application/json" \ -d '{"employee_id":123,"decision":"approved"}'
```

Replace placeholders with the actual deployed URLs and employee ID before testing.

9. Production Readiness Checklist

Area	Status	Note
Modular architecture	Implemented	Pipelines A, B, C and D separated by responsibility.
PostgreSQL state management	Implemented	Employee, task and error-log tables defined.
Duplicate protection	Implemented	Email lookup plus duplicate stop path.
Department routing	Implemented	Engineering → IT; HR/Marketing → HR.
Reusable workers	Implemented	Execute Workflow passes employee context.
Slack approval	Configured	Approval message includes employee details.
Webhook wait/resume	Configured	Approval webhook suffix and two-hour limit present.
Approval persistence	Implemented	PostgreSQL update uses employee_id.
Global error workflow	Implemented	Pipeline D logs failure context.
Credentials	Verify before deploy	Confirm production credentials and secret storage.
Execution retention	Verify before deploy	Set retention/pruning policy for the environment.
cURL E2E test	Run before submit	Capture request, response and resulting DB state.
Evidence screenshots	Prepare final package	Capture master, workers, approval, DB and error evidence.

Deployment note

The workflow exports are the source of truth for node configuration. Before LMS submission, confirm live credentials, error-workflow settings, webhook URLs, database connectivity, and execution-retention policy.

10. LMS Submission Package

Recommended GitHub structure

```
employee-onboarding-automation/ ■■■ workflows/ ■■■ Pipeline A - Master Onboarding Orchestrator.json
■ ■■■ Pipeline B - IT Provision Worker.json ■ ■■■ Pipeline C - HR Provision Worker.json ■■■
database/ ■ ■■■ capstone-db-init.sql ■■■ docs/ ■ ■■■
Employee_Onboarding_Automation_Documentation.pdf ■ ■■■ README.md ■■■ evidence/ ■ ■■■
pipeline-a-execution.png ■ ■■■ pipeline-b-execution.png ■ ■■■ pipeline-c-execution.png ■ ■■■
approval-resume.png ■ ■■■ error-logging.png ■■■ tests/ ■■■ curl-validation.md
```

Required LMS deliverables

- Loom/YouTube demonstration video
- Public GitHub repository
- PDF/DOCX documentation package
- Exported Pipeline A, B and C workflow JSON files
- SQL initialization/schema scripts
- Execution/testing screenshots
- cURL validation commands
- Deployment/setup instructions

Suggested LMS description

Implemented a modular n8n employee onboarding automation using PostgreSQL, reusable IT/HR worker workflows, department routing, Slack manager approval, asynchronous webhook resume, and centralized error logging. Validation focused on duplicate protection, employee/task creation, worker execution, approval-state persistence, and execution-history debugging. The IT and HR provisioning steps are simulated for this assessment. Production deployment requires final credential, webhook, retention, and environment configuration checks.

Appendix — Source & Evidence Notes

This documentation was prepared from the Lesson 5 assessment specification, the supplied n8n workflow exports, and the implementation screenshots provided in the conversation.

- Pipeline A export: webhook ingress, duplicate validation, profile/checklist creation, routing, worker invocation, Slack approval, Wait resume, approval update.
- Pipeline B export: reusable IT worker with employee_id, department and role inputs; it_provisioned update.
- Pipeline C export: reusable HR worker with employee_id, department and role inputs; hr_payroll_setup update.
- Pipeline D export: Error Trigger and centralized capstone_error_log insertion.
- Database initialization export: capstone_employees, capstone_onboarding_tasks and capstone_error_log schema.

Final assessor note

The solution demonstrates a production-oriented modular architecture. Any item marked for verification or final testing should be checked against the live n8n environment immediately before LMS submission.